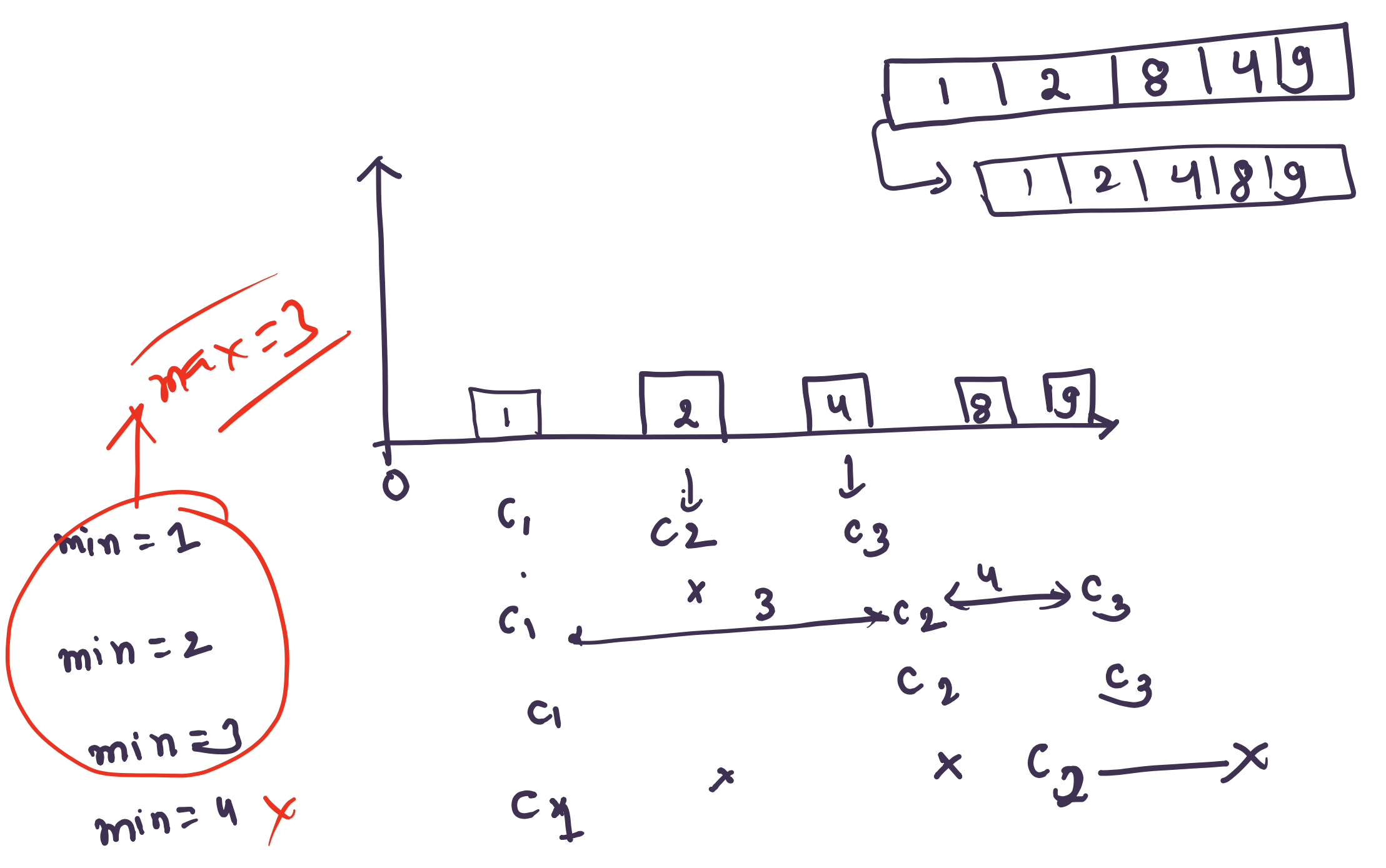
Lec-16
27/10/26 2:04 PM

AGGRCOW - Aggressive cows

#binary-search

Farmer John has built a new long barn, with N (2 ≤ N ≤ 100,000) stalls. The stalls are located along a straight line at positions x1 ... xN (0 ≤ xi ≤ 1,000,000,000).

His C (2 ≤ C ≤ N) cows don't like this barn layout and become aggressive towards each other once put into a stall. To prevent the cows from hurting each other, FJ wants to assign the cows to the stalls, such that the minimum distance between any two of them is as large as possible. What is the largest minimum distance?



①

while (isitpossible (stall, noc, dis) ==) {
dis++
}

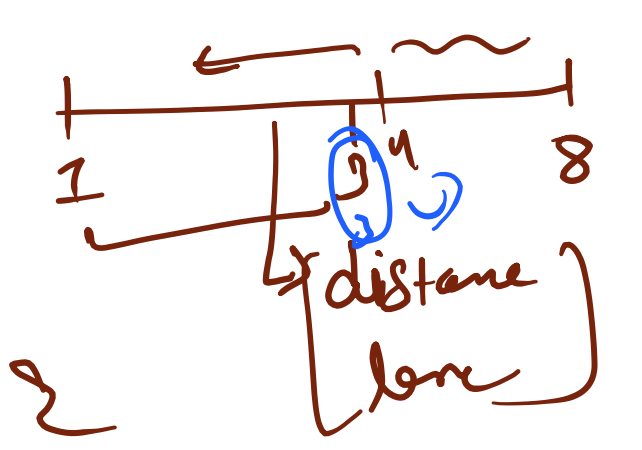
dis-1

[1, 2, 8, 4, 9] → [1, 2, 4, 8, 9]

dis = 4

pos = stall(0)
cow =

for (i=1 i<n i++) {
if (stall[i] - pos >= dis) {
pos = stall[i]
cow++
}
if (noc == cow) {
return true
}
}
false



mid = 4

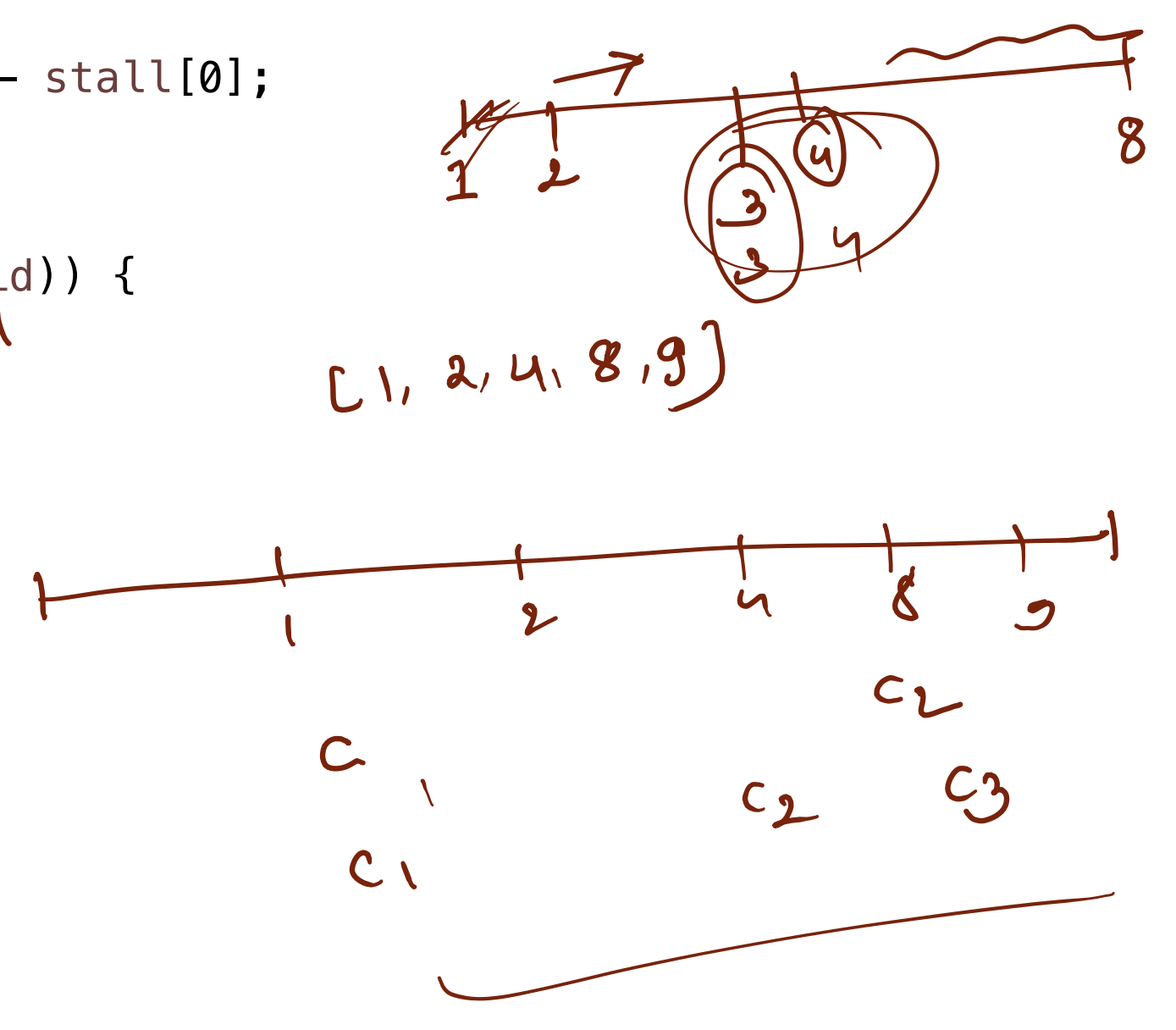
ans

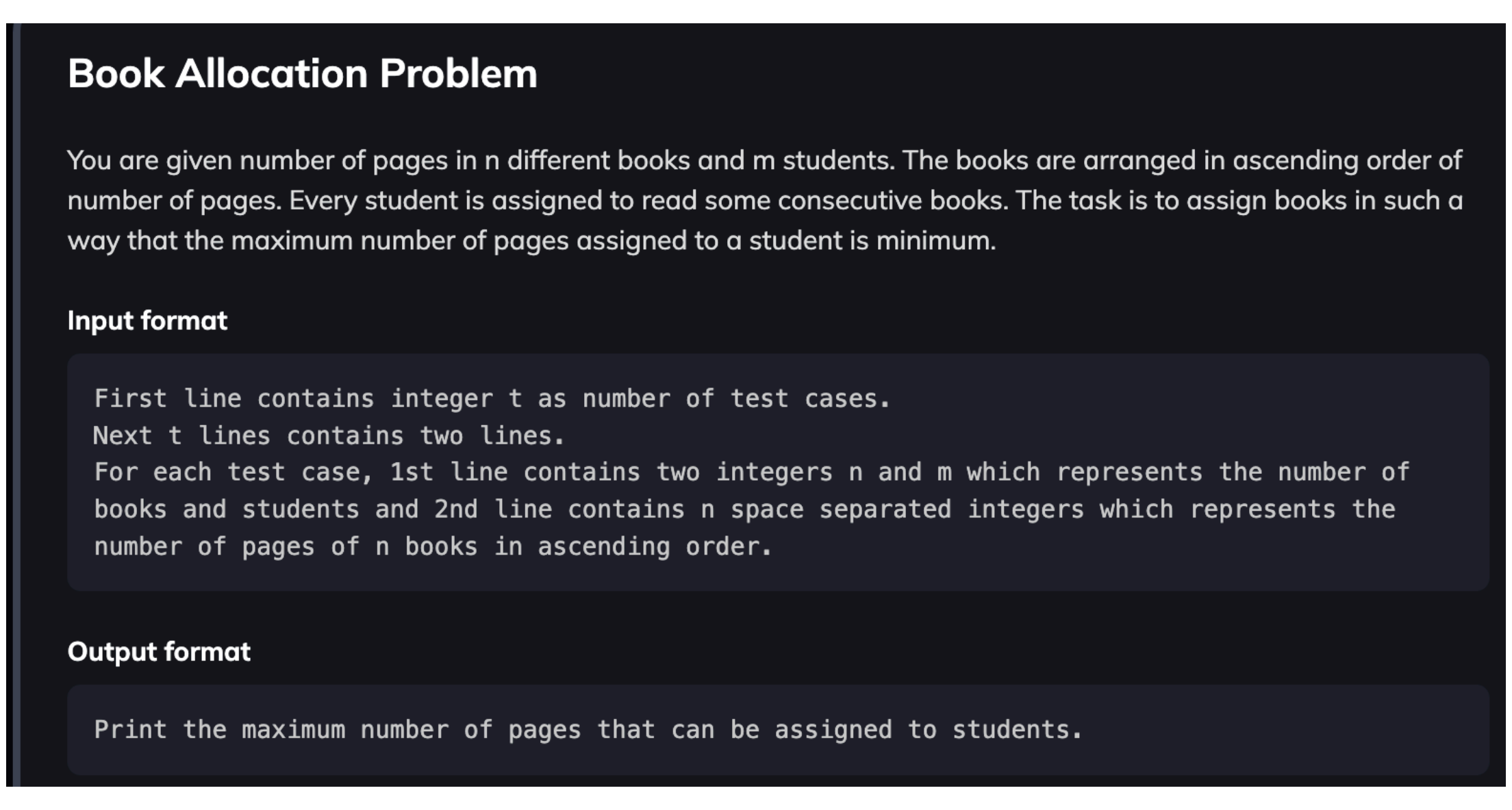
lo = 1
hi = stall (n-1) - stall(0)
while (lo <= hi) {
int mid = (lo+hi)/2
if (isitposs (stall, noc, mid) {
ans = mid
lo = mid+1
}
else
hi = mid-1
}

```
public static int Largest_Minimum(int[] stall, int noc) {
      int lo = 1;
      int hi = stall[stall.length - 1] - stall[0];
      int ans = 0;
      while (lo <= hi) {
         int mid = (lo + hi) / 2;
         if (isitpossible(stall, noc, mid)) {
            ans = mid;
            lo = mid + 1;
         } else {
            hi = mid - 1;
         }

      }
      return ans;

   }
```

ans = ~~2~~ 3

[1, 2, 4, 8, 9]





Book Allocation Problem

You are given number of pages in n different books and m students. The books are arranged in ascending order of number of pages. Every student is assigned to read some consecutive books. The task is to assign books in such a way that the maximum number of pages assigned to a student is minimum.

Input format

```
First line contains integer t as number of test cases.
Next t lines contains two lines.
For each test case, 1st line contains two integers n and m which represents the number of
books and students and 2nd line contains n space separated integers which represents the
number of pages of n books in ascending order.
```

Output format

```
Print the maximum number of pages that can be assigned to students.
```

4 2

[10, 20, 30, 40]

ans = ~~75~~ 60

① ⑪

S1 = 0, S2 = 100 → 100 | S1 = 10, S2 = 90 → 90 | S1 = 30, S2 = 70 → 70 | S1 = 60, S2 = 40 → 60

10, 20
30

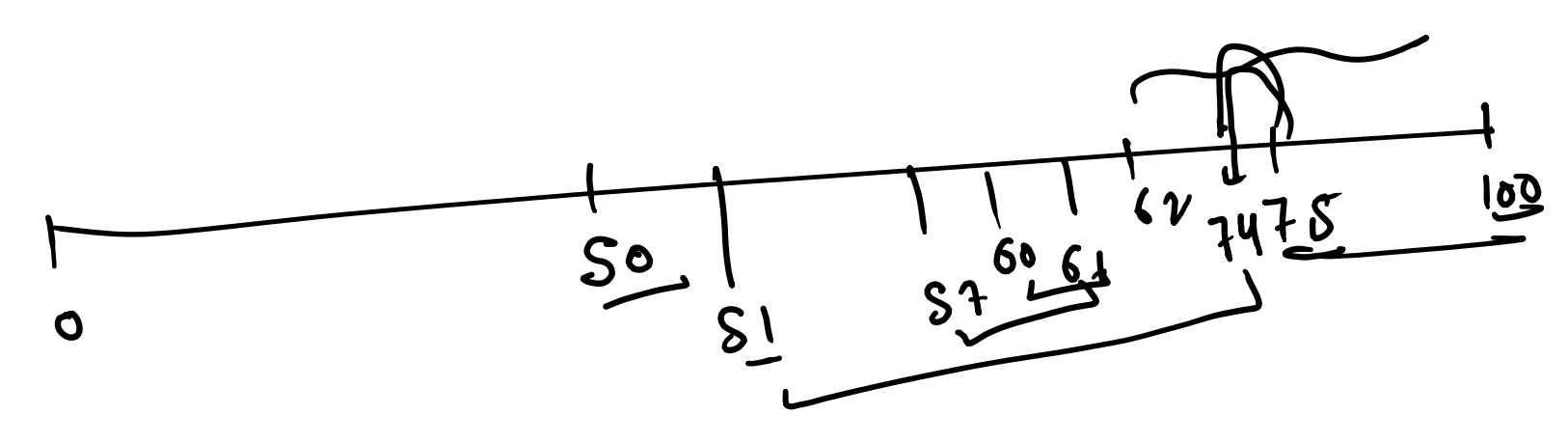
lo = 0
hi = sum

S1 = 10 + 20 = 30 + 30 = 60
S2 = 40 //

ans = 75

[10, 20, 30, 40]

while (lo <= hi) {
int mid = (lo+hi)/2
if (isitPossible(page, nos, mid) {
ans = mid
hi = mid-1
}
else
lo = mid
}



[10, 20, 30, 40]

mid = 62

student = 2

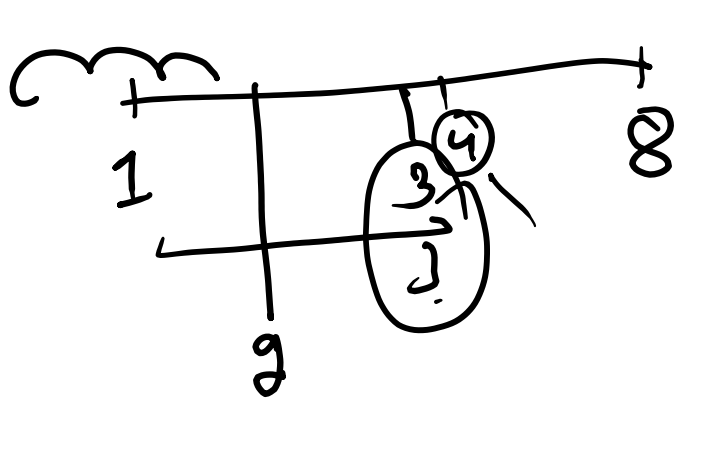
int s = 1
int readpage = 0

while (i < n) {
if (readpage + page[i] <= mid) {
readpage += page[i]
i++
}
else {
s++
readpage = 0
}

```
import java.util.Arrays;
import java.util.Scanner;

public class Aggressive_cows {

   public static void main(String[] args) {
      // TODO Auto-generated method stu
      Scanner sc = new Scanner(System.in);
      int t = sc.nextInt();
      while (t-- > 0) {
         int nos = sc.nextInt();// number of stall
         int noc = sc.nextInt();// number of cows
         int[] stall = new int[nos];
         for (int i = 0; i < stall.length; i++) {
            stall[i] = sc.nextInt();
         }
         Arrays.sort(stall);
         System.out.println(Largest_Minimum(stall, noc));
      }

   }

   public static int Largest_Minimum(int[] stall, int noc) {
      int lo = 1;
      int hi = stall[stall.length - 1] - stall[0];
      int ans = 0;
      while (lo <= hi) {
         int mid = (lo + hi) / 2;
         if (isitpossible(stall, noc, mid)) {
            ans = mid;
            lo = mid + 1;
         } else {
            hi = mid - 1;
         }

      }
      return ans;

   }

   public static boolean isitpossible(int[] stall, int noc, int dis) {
      int pos = stall[0];
      int cow = 1;
      for (int i = 1; i < stall.length; i++) {
         if (stall[i] - pos >= dis) {
            pos = stall[i];
            cow++;
         }
         if (cow == noc) {
            return true;
         }
      }
      return false;
   }
```

ans = ~~2~~ 3

~~[3, 5, 7, 9, 11]~~
[1, 2, 4, 8, 9]



1 → ② → ③ → ④

```
public static void main(String[] args) {
      // TODO Auto-generated method stub
      int[] page = { 10, 20, 30, 40 };
      int student = 2;

   }

   public static int Book_Allocation(int[] page, int student) {
      int lo = 0;
      int hi = 0;
      for (int i = 0; i < page.length; i++) {
         hi = hi + page[i];
      }
      int ans = 0;
      while (lo <= hi) {
         int mid = (lo + hi) / 2;
         if (isitpossible(page, student, mid)) {
            ans = mid;
            hi = mid - 1;
         } else {
            lo = mid + 1;
         }
      }
      return ans;
   }

   public static boolean isitpossible(int[] page, int student, int mid) {
      // TODO Auto-generated method stub
      int s = 1;
      int readpage = 0;
      for (int i = 0; i < page.length;) {
         if (readpage + page[i] <= mid) {
            readpage += page[i];
            i++;
         } else {
            s++;
            readpage = 0;
         }
         if (s > student) {
            return false;
         }
      }
      return true;

   }
```

[10, 20, 30, 40]

nos = 2

S1 = 0, S2 = 100 → 100 | S1 = 10, S2 = 90 → 90 | S1 = 30, S2 = 70 → 70 | S2 = 60, S2 = 40 → 60